Midterm Project — React SPA

Create a complete React Single Page Application using real or simulated data.

Choose your own project idea. Projects should be different.

This is an individual project. Group projects are not allowed.

Requirements

- ☐ Use one data source: an external public API, your own deployed API, or the provided simulated API*.
- ☐ If using your own or simulated API, provide at least 100 initial records for the main list.
- ☐ Use at least 5 meaningful reusable React components.
- ☐ Include list and detail views.
- ☐ Include a controlled form for adding a new item.
- ☐ Implement meaningful search, filtering, sorting, and pagination.
- ☐ Use `useState` and `useEffect` correctly.
- ☐ Handle loading, error, empty, and success states.
- ☐ Use correct effect dependencies and cleanup when needed.
- ☐ Use stable keys and handle component identity correctly.
- ☐ Do not use direct DOM manipulation.
- ☐ Create a clean, consistent, responsive, and usable UI.
- ☐ Deploy the application. The complete application must work from the submitted public link. Working only on localhost is not sufficient.

*The simulated API uses Promises, delays, JSON data, and `localStorage` to behave like a real asynchronous API.

Grading Criteria — 10 points

Project Source Code — 6 points

- React Architecture & Rendering — 1 point
    - Application is divided into meaningful, reusable components.
    - Props and component composition are used correctly.

    - JSX and conditional rendering are clean and understandable.
    - Lists use correct stable keys.
    - Component identity and state preservation/reset are handled correctly where relevant.
- State, Events & Forms — 1.5 points
    - `useState` is used appropriately for application and UI state.
    - Multiple meaningful events are handled.
    - Controlled inputs and forms are implemented correctly.
    - State is updated immutably.
    - State is placed in appropriate components instead of keeping everything in `App`.
    - No unnecessary direct DOM manipulation.
- UI & Consistency — 1.5 points
    - Application has a clear purpose and consistent theme.
    - UI is clean, responsive, and properly styled.
    - No broken, unfinished, placeholder, or unnecessary UI elements.
    - The application provides a coherent user experience rather than unrelated React examples.
- API & Deployment — 2 points
    - API data is meaningfully integrated into the application, not simply fetched and displayed.
    - `useEffect` and dependency arrays are used correctly.
    - Loading, error, empty, and success states are implemented.
    - Effects use cleanup when necessary.
    - Stale async updates and possible race conditions are handled where applicable.
    - Search, filtering, sorting, and pagination work correctly with the chosen data source.
    - The GitHub repository is organized and contains a clear `README`.
    - The submitted deployed application is fully functional.
    - Deployment is mandatory. If the frontend is not deployed, the submitted deployment does not run, or a required custom backend is not deployed, the score for API & Deployment is 0.
    - If your own or simulated API contains fewer than 100 initial records, points will be deducted from this section.

Defense — 4 points

Defense may include theory questions, explanation of your own code, and live modifications.

You should be able to explain:

- component structure and render tree;
- where state is stored and why;
- data flow through props;
- events and controlled components;
- how API data reaches the UI;
- what causes a component to re-render;
- state snapshots and batching;
- reconciliation, component identity, and keys;
- state preservation and reset;
- render vs commit;
- `useEffect` and dependency arrays;
- effect cleanup;
- stale closures and race conditions;
- loading, error, empty, and success states.

You may also be asked to modify your application during the defense, for example:

- add another filter;
- add a controlled input;
- display another API field;
- add a reset action;
- change an effect dependency and explain the result;
- add or modify sorting;
- add a loading/error condition;
- modify an event handler.

Submission

Submit:

1. GitHub repository link
2. Deployed application link
3. Press "turn in" button BEFORE the deadline. The assignment must be submitted before the deadline. No late submissions allowed. Missing submission receives 0 points for the Project Source Code part (out of 6 points).